Embedded System

Francesco De Patre – 382760

Introduzione

Il quarto laboratorio ha come obiettivo la configurazione e l'utilizzo di meccanismi di output e debugging.

Sono stati utilizzati due approcci distinti: la stampa diretta su terminale tramite la funzione printk() e l'utilizzo del modulo di logging integrato.

A differenza della programmazione tradizionale, nei sistemi embedded le operazioni di output non sono solo uno strumento di debug, ma rappresentano un compromesso tra visibilità dello stato del sistema e consumo di risorse limitate, come memoria e tempo di esecuzione.

Metodologia

Per lo svolgimento del laboratorio è stato seguito il materiale della Nordic Developer Academy come riferimento principale, integrando le indicazioni con test pratici sul dispositivo e osservazione diretta dell'output seriale.

L'approccio adottato è stato di tipo iterativo: dopo una prima esecuzione degli esempi forniti, sono state effettuate modifiche al codice e alla configurazione del sistema per analizzare il comportamento dei diversi metodi di output.

In particolare, sono stati confrontati gli effetti dell'utilizzo di printk() rispetto al modulo di logging, variando i livelli di log e osservando le differenze in termini di leggibilità e controllo.

Il debug è stato effettuato principalmente tramite terminale seriale, sfruttando l'output UART del dispositivo.

Sezione LAB

Obiettivo tecnico

L'obiettivo del laboratorio è comprendere i meccanismi di output nei sistemi embedded confrontando l'approccio diretto tramite printk() con il sistema di logging avanzato fornito da Zephyr, e analizzando le implicazioni in termini di efficienza e gestione delle risorse.

Cosa è stato fatto durante il laboratorio

Durante il laboratorio sono stati implementati semplici esempi di stampa su terminale utilizzando printk(), seguiti dall'utilizzo del modulo di logging per produrre output strutturato.

Successivamente, sono stati modificati i parametri di configurazione del logger per controllare il livello di dettaglio dei messaggi, fino a limitare l'output ai soli messaggi di errore tramite configurazione nel file di progetto.

Cosa si può capire

Questo laboratorio ha evidenziato come nei sistemi embedded la gestione dell'output sia un aspetto non banale.

In particolar modo la funzione printk() rappresenta un modo semplice ed immediato, ma poco flessibile e non adatto a progetti complessi mentre il modulo di logging introduce un livello di astrazione che consente di filtrare i messaggi per livello di severità, abilitare/disabilitare il logging senza modificare il codice, migliorare la leggibilità tramite formattazione e timestamp.

È emerso inoltre che il logging ha un impatto sulle risorse del sistema (memoria e tempo di esecuzione), rendendo necessario un bilanciamento tra quantità di informazioni e overhead introdotto.

Questo riflette un aspetto fondamentale dello sviluppo embedded: ogni funzionalità, anche apparentemente semplice come la stampa su terminale, deve essere valutata in termini di costo computazionale.

Problemi riscontrati

Uno dei principali problemi riscontrati riguarda la configurazione del logger, che non è immediata e richiede la modifica corretta dei parametri nel file di configurazione del progetto.

Inoltre, è stato necessario comprendere la differenza tra configurazione a compile-time e runtime, in quanto alcune modifiche non avevano effetto senza una ricompilazione completa (pristine build).

Miglioramenti possibili

Un possibile miglioramento consiste nell'utilizzare configurazioni dinamiche del logging per adattare il livello di dettaglio in base alla fase di sviluppo

Infine, l'integrazione del logging con strumenti di debugging più avanzati potrebbe migliorare ulteriormente l'analisi del comportamento del sistema.

Evidenze

Le seguenti immagini si riferiscono all'output della stessa funzione (calcolo del fattoriale di un numero I = 1...10) rispettivamente a:

- Esempio 1 (stampa semplice con la funzione printk)
- Esempio 2 (stampa con il modulo logger)
- Esempio 3 (stampa con il modulo logger con configurazione personalizzata)

USB-DEV
16BEDDB16139A2C3

TERMINAL SETTINGS ABOUT

SERIAL PORT

/dev/tty.usbmodem1101

Disconnect from port

SERIAL SETTINGS

TERMINAL MODE

WRITE TO FILE

> Type and press Enter to send

Send

```
The factorial of  4 = 24
The factorial of  5 = 120
The factorial of  6 = 720
The factorial of  7 = 5040
The factorial of  8 = 40320
The factorial of  9 = 362880
The factorial of 10 = 3628800
_____________________________________________________
Calculating the factorials of numbers from 1 to 10:
The factorial of  1 = 1
The factorial of  2 = 2
The factorial of  3 = 6
The factorial of  4 = 24
The factorial of  5 = 120
The factorial of  6 = 720
The factorial of  7 = 5040
The factorial of  8 = 40320
The factorial of  9 = 362880
The factorial of 10 = 3628800
_____________________________________________________
Calculating the factorials of numbers from 1 to 10:
The factorial of  1 = 1
The factorial of  2 = 2
The factorial of  3 = 6
The factorial of  4 = 24
The factorial of  5 = 120
The factorial of  6 = 720
The factorial of  7 = 5040
The factorial of  8 = 40320
The factorial of  9 = 362880
The factorial of 10 = 3628800
_____________________________________________________
```

CLEAR CONSOLE

USB-DEV
16BEDDB16139A2C3

TERMINAL SETTINGS ABOUT

SERIAL PORT

/dev/tty.usbmodem1101

Disconnect from port

SERIAL SETTINGS

TERMINAL MODE

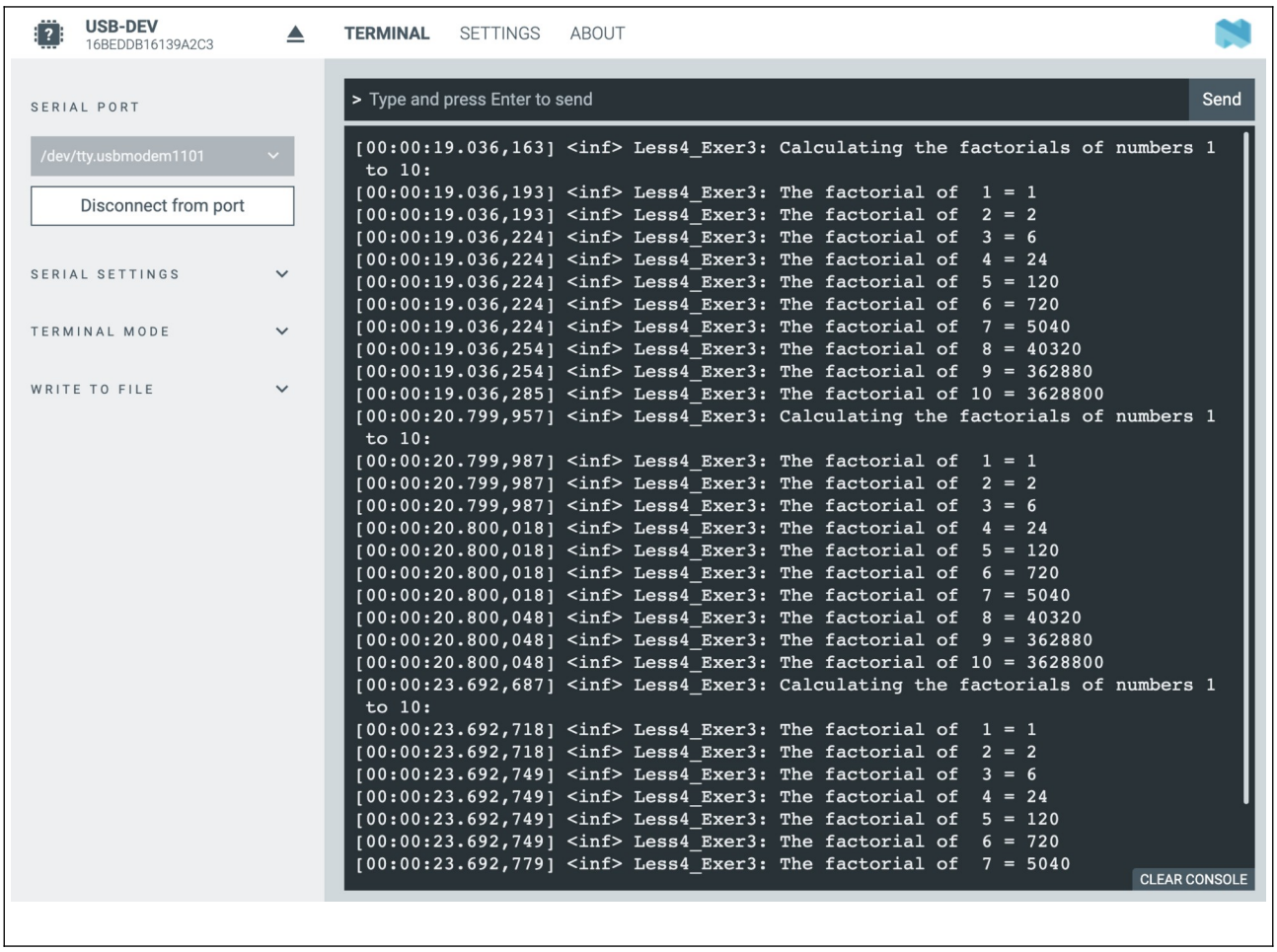
WRITE TO FILE

> Type and press Enter to send

Send

```
[00:00:00.000,854] <inf> Less4_Exer2: nRF Connect SDK Fundamentals
[00:00:00.000,854] <inf> Less4_Exer2: Exercise 2
[00:00:00.000,854] <dbg> Less4_Exer2: main: A log message in debug level
[00:00:00.000,854] <wrn> Less4_Exer2: A log message in warning level!
[00:00:00.000,885] <err> Less4_Exer2: A log message in Error level!
[00:00:00.000,885] <inf> Less4_Exer2: Sample Data!
                                      00 01 02 03 04 05 06 07  48 65 6c 6c 6f
          |........ Hello
[00:00:21.017,791] <inf> Less4_Exer2: Calculating the factorials of numbers 1
 to 10:
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  1 = 1
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  2 = 2
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  3 = 6
[00:00:21.017,822] <inf> Less4_Exer2: The factorial of  4 = 24
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  5 = 120
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  6 = 720
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  7 = 5040
[00:00:21.017,852] <inf> Less4_Exer2: The factorial of  8 = 40320
[00:00:21.017,883] <inf> Less4_Exer2: The factorial of  9 = 362880
[00:00:21.017,883] <inf> Less4_Exer2: The factorial of 10 = 3628800
[00:00:25.224,914] <inf> Less4_Exer2: Calculating the factorials of numbers 1
 to 10:
[00:00:25.224,945] <inf> Less4_Exer2: The factorial of  1 = 1
[00:00:25.224,945] <inf> Less4_Exer2: The factorial of  2 = 2
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  3 = 6
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  4 = 24
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  5 = 120
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  6 = 720
[00:00:25.224,975] <inf> Less4_Exer2: The factorial of  7 = 5040
[00:00:25.225,006] <inf> Less4_Exer2: The factorial of  8 = 40320
[00:00:25.225,006] <inf> Less4_Exer2: The factorial of  9 = 362880
[00:00:25.225,036] <inf> Less4_Exer2: The factorial of 10 = 3628800
```

CLEAR CONSOLE



Task

Per filtrare l'output del logger e visualizzare esclusivamente i messaggi di errore, è necessario intervenire sulla configurazione del sistema di logging a livello di progetto.

In particolare, il comportamento del logger può essere controllato tramite il file di configurazione prj.conf, che permette di sovrascrivere le impostazioni definite nel codice sorgente. Per ottenere il filtraggio desiderato, è stata aggiunta la seguente direttiva:

```
CONFIG_LOG_OVERRIDE_LEVEL=1
```

Questa configurazione forza il livello globale del logger al livello *error*, indipendentemente da quanto specificato nel codice tramite macro di logging (come LOG_INF, LOG_DBG, ecc.). Di conseguenza, tutti i messaggi con severità inferiore (informativi o di debug) vengono automaticamente ignorati.

GenAI Statement

Durante lo svolgimento dei laboratori e la redazione del presente report sono stati utilizzati strumenti di Generative AI come supporto allo studio e alla scrittura.

In particolare, tali strumenti sono stati impiegati per:

- migliorare la qualità espositiva del report, rendendo il testo più chiaro, strutturato e coerente;
- facilitare la comprensione della documentazione tecnica relativa ai diversi moduli del corso (ad esempio nRF Connect SDK e Toolchain);
- supportare la traduzione e riformulazione dei contenuti, al fine di ottenere una terminologia tecnica più precisa;
- assistere nella risoluzione di problemi pratici, in particolare legati alla compatibilità tra il sistema utilizzato (computer personale), gli strumenti software e la board utilizzata nei laboratori.

Tutti i contenuti generati con il supporto di strumenti di GenAI sono stati revisionati criticamente, verificati e adattati sulla base della comprensione personale degli argomenti trattati.